

Can Autonomous AI Execution Become Provable?

EGA V8: A Five-Layer Execution Governance Architecture for Replayable, Auditable, and Containable Autonomous AI Agents

Abstract

Autonomous AI agents increasingly perform actions with financial, operational, and legal consequences through interactions with tools, services, and external environments. As these systems evolve beyond passive text generation into autonomous workflow execution, maintaining workflow integrity becomes a critical governance challenge. Existing safety approaches primarily focus on model behavior and token-level controls, providing limited visibility into how workflow actions evolve after execution has begun. This paper presents EGA v8 (Execution Governance AI), an execution-governance framework designed to make autonomous AI workflows replayable, traceable, auditable, and governable throughout the execution lifecycle. The framework introduces a five-layer architecture consisting of Trinity Core, Provenance Layer, Containment Layer, Workflow Interface Layer, and Safe-Test Runtime. Together, these components enable deterministic replay, provenance-aware reconstruction, execution-divergence detection, trust-state evaluation, and fail-closed containment.

To evaluate the framework, we conducted replay-validation, provenance-validation, containment-validation, detection-quality, live-LLM, large-scale, and hard-case evaluations using adversarial workflow scenarios inspired by Mythos-like execution-mutation attacks. Experimental results demonstrated 100% replay consistency, 100% provenance reconstruction consistency, 100% containment success, and 100% detection performance across large-scale and hard-case evaluation campaigns, while maintaining 0% false-positive and false-negative rates. The findings suggest that replayability, provenance reconstruction, and execution-level validation can serve as practical governance mechanisms for maintaining workflow integrity in autonomous AI systems. More broadly, this work argues that execution governance represents a distinct architectural layer that complements model safety and system security, providing a foundation for trustworthy autonomous-agent ecosystems.

1 Introduction

Autonomous AI agents are increasingly transitioning from passive language-generation systems into active execution systems capable of interacting with tools, services, APIs, and external environments. Modern agentic architectures can initiate transactions, orchestrate workflows, manipulate digital infrastructure, and perform operational tasks with limited human intervention. As these systems acquire greater autonomy, ensuring the integrity of workflow execution becomes a critical challenge for trustworthy AI deployment.

This transition fundamentally changes the scope of AI governance. Traditional AI-safety approaches primarily focus on model behavior through alignment techniques, prompt filtering, output moderation, and token-level safety controls. While these mechanisms remain important, they provide limited visibility into how workflow actions evolve once execution has begun. In autonomous-agent environments, failures frequently emerge not from individual model outputs, but from interactions among reasoning processes, tool invocations, workflow transitions, and external systems.

Recent advances in agent-runtime frameworks have amplified this challenge. Contemporary agent architectures integrate large language models with retrieval systems, memory stores, browser automation, external APIs, and transaction-execution services. Although these capabilities improve autonomy, they also introduce execution pathways that may diverge from intended workflow behavior. Existing observability and monitoring systems primarily capture logs, traces, or telemetry after execution has occurred. Consequently, they

provide limited support for determining whether workflow behavior remained consistent with governance expectations throughout execution.

This limitation becomes particularly significant in high-trust environments where autonomous agents possess financial, operational, or legal authority. Once workflow execution diverges from approved behavior, unsafe actions may propagate rapidly across interconnected systems. Existing approaches generally lack mechanisms for deterministically reconstructing execution histories, identifying causal execution divergence, or enforcing governance controls before irreversible actions occur.

This research addresses this challenge through EGA v8 (Execution Governance AI), a framework designed to make autonomous AI execution replayable, traceable, auditable, and governable throughout the execution lifecycle. Rather than treating governance as a post-execution activity, EGA v8 introduces execution-layer governance directly into runtime operations through deterministic replay, provenance-aware reconstruction, trust-state evaluation, and fail-closed containment.

The proposed architecture consists of five runtime layers:

1. **Trinity Core** — deterministic replay and execution-divergence validation
2. **Provenance Layer** — causal reconstruction and execution attribution
3. **Containment Layer** — trust escalation and fail-closed enforcement
4. **Workflow Interface Layer** — governance visualization and operator interaction
5. **Safe-Test Runtime** — adversarial evaluation and continuous governance testing

Together, these layers establish a governance framework capable of continuously evaluating workflow integrity before unsafe execution propagates into irreversible outcomes.

This study evaluates EGA v8 through replay-validation, provenance-validation, containment-validation, detection-quality, live-LLM, large-scale, and hard-case evaluation campaigns. Experimental results demonstrate deterministic replay consistency, provenance reconstruction consistency, containment effectiveness, and robust detection performance across adversarial workflow scenarios inspired by Mythos-like execution-mutation attacks.

More broadly, this work argues that replayability, provenance reconstruction, and execution-level validation represent foundational governance mechanisms for autonomous AI systems. As agent ecosystems continue to scale toward increasingly autonomous operation, workflow integrity can no longer be implicitly assumed. Instead, autonomous AI execution must become traceable, reproducible, auditable, and ultimately provable.

2 Methodology

2.1 Threat Model

2.1.1 Adversary Assumptions

Unsafe agent behaviors may emerge from workflow-level failures—including execution divergence, unauthorized tool usage, parameter mutation, governance bypass, and unsafe action propagation—even when the underlying language model operates as intended. A representative example is the Mythos-style attack, in which an apparently legitimate workflow evolves into an unsafe execution path through manipulated tool interactions and governance failures without requiring compromise of model weights or infrastructure. Accordingly, we model an adversary with workflow-level visibility and the ability to influence execution through:

- modification of workflow parameters,
- unauthorized tool invocation or substitution,
- manipulation of execution states or workflow transitions,
- governance-policy evasion, and

- containment-evasion attempts.

The adversary is assumed to lack access to model weights, cryptographic secrets, operating-system privileges, and infrastructure-level administrative controls. The primary objective of the adversary is to propagate unsafe workflow actions while preserving the appearance of legitimate execution. Therefore, EGA v8 is evaluated on its ability to detect execution anomalies, reconstruct execution provenance, escalate trust states, and enforce fail-closed containment before unsafe actions propagate beyond the runtime boundary.

2.1.2 Attack Taxonomy and Threat Table

To evaluate runtime-governance robustness, we define five categories of execution-layer attacks summarized in Table 1.

Table 1. AI Agent Attack Scenarios

Attack Scenario	Category	Target Component
Quantity Mutation	Integrity (A)	Tool Parameters
Fake Storefront	Integrity (A)	Commerce Workflow
Unauthorized Purchase	Authorization (B)	Execution Control
Tool Injection	Tool Manipulation (C)	Runtime Interface
Policy Bypass	Policy Evasion (D)	Governance Layer
Containment Bypass	Containment (E)	Containment Layer

Category A – Integrity Attacks modify workflow parameters or execution states while preserving the appearance of legitimate execution.

Category B – Authorization Attacks attempt to execute actions without satisfying required authorization constraints.

Category C – Tool Manipulation Attacks exploit or redirect tool invocations to produce unauthorized outcomes.

Category D – Policy Evasion Attacks attempt to bypass governance policies, approval requirements, or runtime safeguards.

Category E – Containment Attacks attempt to override, disable, or circumvent containment mechanisms after anomalous behavior has been detected.

Together, these attack classes represent common workflow-level failure modes in autonomous-agent systems, including execution mutation, unauthorized actions, tool misuse, governance evasion, and containment failure.

2.1.3 Attack Generation

Adversarial workflows were generated using deterministic attack templates to ensure reproducibility and controlled evaluation. Templates systematically mutated workflow parameters, tool selections, policy states, or containment conditions while preserving overall workflow structure. This automated generation process enabled reproducible evaluation campaigns ranging from small-scale experiments to stress-testing scenarios containing up to 10,000 workflows.

2.1.4 Scope and Limitations

This work focuses exclusively on execution-governance failures occurring within autonomous workflow runtimes. The following attack classes are considered outside the scope of this study:

- operating-system compromise,
- network-level attacks,

- hardware attacks,
- model-weight extraction,
- training-time poisoning, and
- infrastructure-level privilege escalation.

Accordingly, EGA v8 should be interpreted as an execution-governance framework rather than a comprehensive cybersecurity solution. Its objective is to provide replayable, auditable, and containable workflow execution for autonomous AI agents.

2.2 EGA v8 Architecture

2.2.1 Architectural Overview

Figure 1 illustrates the EGA v8 runtime governance architecture. The framework consists of five governance layers: Trinity Core, Provenance Layer, Containment Layer, Workflow Interface Layer, and Safe-Test Runtime. Unlike conventional agent runtimes that primarily focus on task execution, EGA v8 treats execution governance as a first-class objective. Workflow transitions, tool invocations, provenance events, and containment decisions are continuously recorded as governance artifacts that support replay, auditing, and containment.

The Trinity Core provides deterministic replay, the Provenance Layer reconstructs execution lineage, the Containment Layer enforces fail-closed governance, the Workflow Interface Layer governs external interactions, and the Safe-Test Runtime supports adversarial evaluation and reproducible experimentation. Together, these components transform autonomous-agent execution into a replayable, auditable, and containable process.

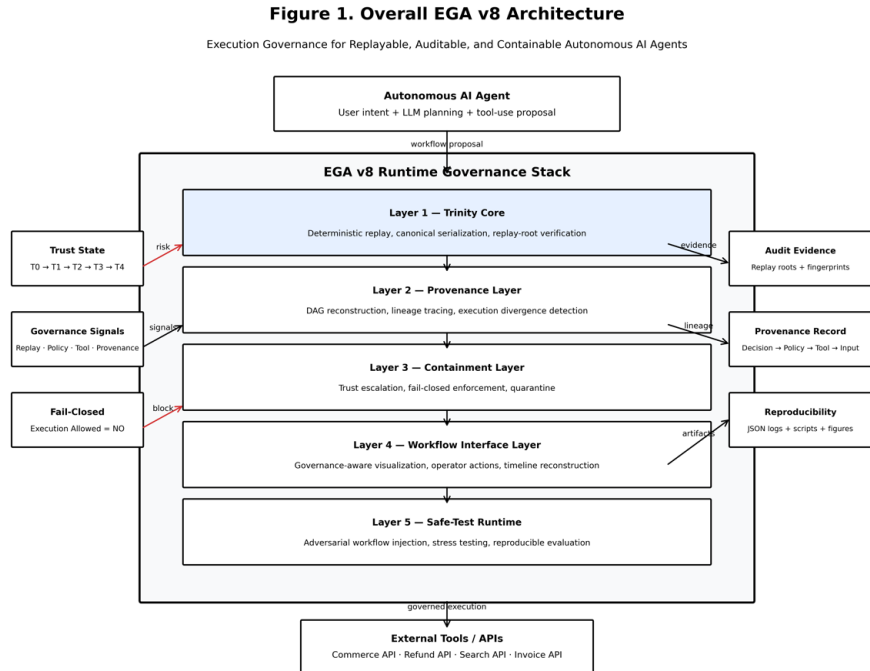


Figure 1: Overview of the EGA v8 runtime governance architecture. Autonomous workflows traverse five governance layers that provide deterministic replay, provenance reconstruction, trust escalation, and fail-closed containment.

Figure 1: Overall EGA v8 architecture.

2.2.2 Trinity Core

Key Contribution: Deterministic replay and execution-state reconstruction.

The Trinity Core serves as the governance backbone of EGA v8. It captures workflow states, execution transitions, and tool-call events to enable deterministic replay and historical state reconstruction. Replay records are subsequently used for provenance analysis, trust escalation, and containment decisions.

Replay Consistency Definition Let a workflow execution trace be represented as:

$$W = (s_0, a_1, s_1, \dots, a_n, s_n)$$

where s_i denotes workflow states and a_i denotes execution actions. The replay function reconstructs an observed execution:

$$R(W) = W'$$

Replay consistency is defined as:

$$RC(W) = \begin{cases} 1, & \text{if } W = W' \\ 0, & \text{if } W \neq W' \end{cases}$$

Execution divergence is detected whenever:

$$RC(W) = 0$$

2.2.3 Provenance Layer

Key Contribution: Provenance reconstruction and divergence detection.

The Provenance Layer converts runtime events into a structured execution graph representing workflow causality and tool dependencies. The layer continuously evaluates replay outcomes and identifies execution divergence. Provenance-aware alerts are forwarded to the governance engine for further trust evaluation and containment actions.

Provenance Graph Definition Execution provenance is modeled as a directed graph:

$$G = (V, E)$$

where:

- V represents workflow states, tool invocations, policy decisions, and containment events;
- E represents causal dependencies between execution events.

A provenance path between two events is defined as:

$$P(v_i, v_j) = 1$$

if a valid causal path exists between v_i and v_j , and

$$P(v_i, v_j) = 0$$

otherwise.

Actions lacking a valid provenance path are treated as governance anomalies.

2.2.4 Containment Layer

Key Contribution: Fail-closed governance and unsafe-action isolation.

The Containment Layer prevents unsafe workflow propagation after anomaly detection. When governance violations exceed predefined thresholds, workflows transition into a containment state. Available actions include execution suspension, tool-call blocking, workflow isolation, and escalation to human review. EGA v8 adopts a fail-closed strategy in which suspicious workflows remain blocked until governance conditions are satisfied.

2.2.5 Workflow Interface Layer

Key Contribution: Governance-aware external interaction.

The Workflow Interface Layer acts as the operational boundary between autonomous agents and external services. All tool invocations and execution requests pass through this layer, where provenance identifiers, trust states, containment status, and policy-validation results are attached to runtime actions.

2.2.6 Safe-Test Runtime

Key Contribution: Controlled adversarial evaluation.

The Safe-Test Runtime provides an isolated environment for evaluating autonomous workflows under both normal and adversarial conditions. The runtime supports attack injection, replay validation, containment testing, and large-scale stress testing while maintaining experimental reproducibility.

2.2.7 Trust Escalation Lifecycle

Key Contribution: Risk-adaptive governance transitions.

Workflow execution begins in a normal trust state (T0). As replay divergence, provenance inconsistencies, or policy violations accumulate, governance states escalate progressively (T1–T4). Higher trust states activate increasingly restrictive controls, including enhanced verification, containment preparation, and execution blocking. The final state represents active containment, preventing unsafe workflow propagation through fail-closed governance.

2.3 Evaluation Protocol

To evaluate EGA v8 under both normal and adversarial operating conditions, we designed a multi-stage evaluation framework addressing the following research questions:

- **RQ1 (Deterministic Replay):** Can EGA v8 reliably reconstruct workflow execution through deterministic replay?
- **RQ2 (Anomaly Detection):** Can EGA v8 accurately detect execution divergence and governance-policy violations?
- **RQ3 (Fail-Closed Containment):** Can EGA v8 prevent unsafe workflow propagation through fail-closed containment mechanisms?
- **RQ4 (Hard-Case Robustness):** Does EGA v8 maintain detection quality when adversarial workflows closely resemble legitimate execution?
- **RQ5 (Live LLM Generalization):** Can EGA v8 enforce governance effectively against adversarial workflows generated by frontier language models?

2.3.1 Baseline Systems

EGA v8 was compared against three baseline runtime configurations:

- **Baseline A (Execution-Only):** Executes workflows without replay validation, provenance tracking, or containment.
- **Baseline B (Logging Runtime):** Records execution events but lacks replay validation, trust escalation, and fail-closed enforcement.
- **Baseline C (Coordination Runtime):** Supports multi-agent orchestration without provenance-aware replay verification or mandatory containment.

These baselines progressively increase runtime capability while intentionally excluding the governance mechanisms introduced by EGA v8.

2.3.2 Workflow Suite

The evaluation suite includes both normal and adversarial workflows.

Normal workflows consist of purchase, refund, search, and tool-execution tasks.

Adversarial workflows include:

- Quantity Mutation
- Unauthorized Purchase
- Fake Storefront
- Tool Injection
- Policy Bypass
- Containment Bypass

To evaluate boundary conditions, additional hard-case workflows were constructed in which malicious execution closely resembles legitimate workflow behavior.

Experiments were conducted at scales ranging from (10^2) to (10^4) workflow executions.

2.3.3 Runtime Governance Metrics

Governance effectiveness was evaluated using the following runtime-governance metrics.

Replay Rate; $\mathcal{R}_{rep} = \frac{N_{reconstructed}}{N_{total}}$

where $N_{reconstructed}$ denotes successfully reconstructed workflow traces and N_{total} denotes total workflow executions.

Detection Rate; $\mathcal{R}_{det} = \frac{N_{detected}}{N_{attack}}$

where $N_{detected}$ denotes correctly detected governance violations and N_{attack} denotes total adversarial workflows.

Escalation Rate; $\mathcal{R}_{esc} = \frac{N_{escalated}}{N_{detected}}$

where $N_{escalated}$ denotes anomalies that successfully triggered trust-state escalation.

Containment Success Rate; $\mathcal{R}_{con} = \frac{N_{contained}}{N_{unsafe}}$

where $N_{contained}$ denotes unsafe workflows successfully isolated before propagation and N_{unsafe} denotes total unsafe workflows.

Unsafe Propagation Rate; $\mathcal{R}_{prop} = \frac{N_{propagated}}{N_{unsafe}}$

where $N_{propagated}$ denotes unsafe workflows that bypassed governance controls and continued execution.

Additional operational metrics include latency overhead measured using P_{50} , P_{90} , and P_{99} latency statistics, independently of underlying language-model inference time.

Together, these metrics provide a quantitative characterization of replay correctness, governance effectiveness, trust-state responsiveness, containment performance, unsafe-propagation prevention, and operational overhead.

2.3.4 Detection Quality Metrics

Detection quality was evaluated using confusion-matrix analysis based on:

- True Positive (TP)
- True Negative (TN)
- False Positive (FP)
- False Negative (FN)

From these values, we compute the following metrics:

$$Precision = \frac{TP}{TP+FP}, Recall = \frac{TP}{TP+FN}$$

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

$$FPR = \frac{FP}{FP+TN}, FNR = \frac{FN}{TP+FN}$$

Detection-quality experiments were conducted at workflow scales of 100, 1,000, and 10,000 executions, together with a dedicated hard-case evaluation campaign.

2.3.5 Live LLM Adversarial Evaluation

To reduce synthetic-evaluation bias, EGA v8 was additionally evaluated using outputs generated by multiple frontier language models. The objective was to determine whether governance controls remain effective when adversarial workflows originate from open-ended language-model reasoning rather than deterministic attack templates. For each evaluation scenario, detection success, containment success, execution blocking, and unsafe-propagation outcomes were measured.

2.3.6 Stress-Test Campaigns

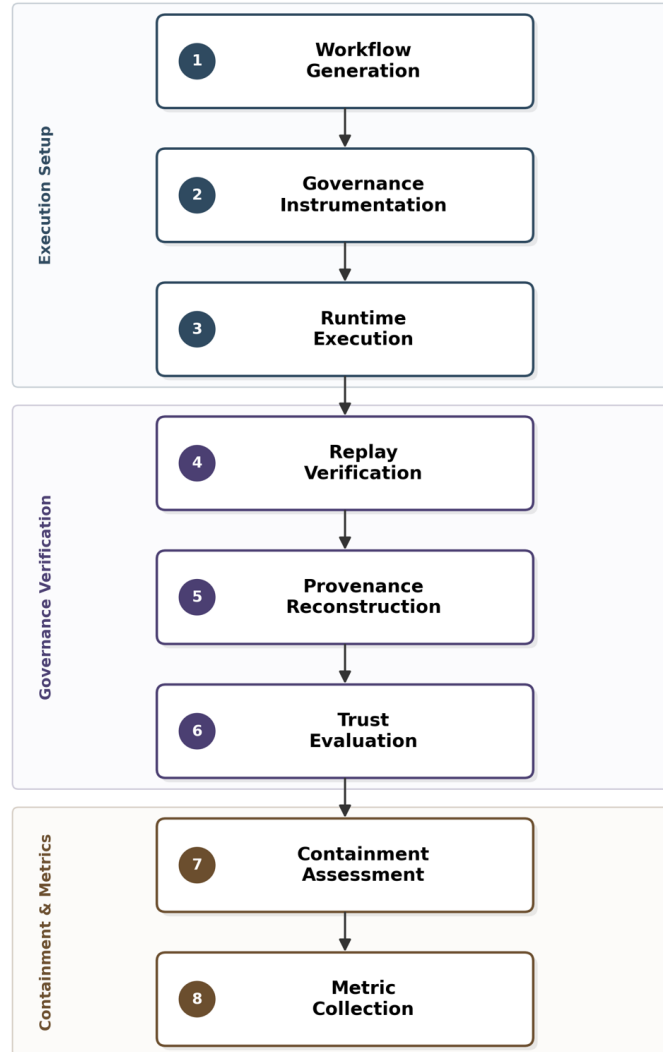
To evaluate operational robustness at scale, EGA v8 was subjected to large-scale stress-testing campaigns containing adversarial workflow mutations and governance-violation scenarios. The evaluation measures replay consistency, provenance-graph stability, trust-escalation responsiveness, containment latency, and governance scalability under load. The objective is to verify that governance mechanisms remain effective and operationally viable under realistic runtime conditions.

2.4 Reproducibility

To support transparent and independently verifiable evaluation, EGA v8 was developed and assessed using a deterministic and isolated experimental framework. All workflow-generation procedures, runtime configurations, evaluation protocols, and analysis pipelines were designed to facilitate independent reproduction of the reported results.

Figure 2. Deterministic Governance Evaluation Pipeline of EGA v8

Standardized execution path for reproducible replay, provenance, containment, and metric generation



Each trial follows the same deterministic pipeline to support independent reproduction of reported results.

Figure 2: Deterministic governance evaluation pipeline of EGA v8.

2.4.1 Controlled Evaluation Environment

All benchmark campaigns were executed within containerized and resource-isolated environments to minimize external sources of variance and ensure consistent execution behavior. Deterministic evaluation was enforced through the following mechanisms:

- **Fixed Pseudo-Random Seeds:** All stochastic processes involved in workflow generation, attack injection, template mutation, and language-model interaction utilize explicit fixed pseudo-random seeds.
- **Standardized Execution Pipeline:** Every workflow follows an identical evaluation pipeline:

3 Results

3.1 Replay Validation

Replay validation was performed by reconstructing workflow executions from the replay artifacts generated by the Trinity Core and comparing the reconstructed traces with the original execution histories. Table 2 summarizes the replay-validation results. Across all evaluated workflows, replay reconstruction was completed successfully, yielding a replay rate of 100%. Replay-root fingerprints remained identical between original and reconstructed executions, and no replay inconsistencies were observed.

The results were consistent across both normal and adversarial workflow scenarios. No workflow produced replay divergence during the validation process. In addition, workflow fingerprints remained stable across repeated executions, resulting in a fingerprint-stability rate of 100%. Overall, all evaluated workflows were reconstructed successfully under the experimental conditions, with identical replay roots and execution fingerprints across repeated re-executions.

Table 2. Replay Validation Results Across Deterministic Workflow Re-Executions

Workflow	Replay Count	Replay Consistency Rate	Replay Root Match Rate	Workflow Fingerprint Stability
WF-001 (Purchase)	100	100%	100%	100%
WF-002 (Refund)	100	100%	100%	100%
Overall	200	100%	100%	100%

3.2 Provenance Validation

Provenance validation was performed by reconstructing workflow execution lineage from runtime events recorded during workflow execution. Figure 3 illustrates an example provenance graph reconstructed from an evaluated workflow. As shown in Figure 3, workflow execution was represented as a directed acyclic graph (DAG), where nodes correspond to workflow states, tool invocations, and governance events, and edges represent causal execution relationships. For normal workflows, all execution transitions were reconstructed successfully, producing a complete provenance path from workflow initiation to execution completion.

For adversarial workflows, provenance reconstruction identified execution paths containing unauthorized tool invocations and workflow mutations. In these cases, divergence points were recorded within the reconstructed provenance graph, enabling anomalous execution paths to be

FIGURE 3. PROVENANCE DAG RECONSTRUCTION EXAMPLE

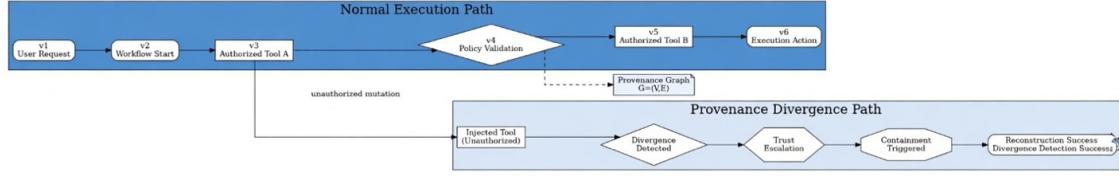


Figure 3: Provenance DAG reconstruction example.

distinguished from the original workflow lineage. Across all evaluated workflows, provenance reconstruction completed successfully without graph-generation failures. In addition, all injected attack scenarios produced identifiable divergence points within the reconstructed execution graph. No provenance records were lost during evaluation, and complete execution lineage remained available for post-execution inspection.

These results demonstrate that workflow execution histories were successfully reconstructed as provenance graphs under both normal and adversarial operating conditions.

3.3 Containment Validation

Figure 4 illustrates the trust-escalation and containment lifecycle observed during runtime-governance evaluation. Workflows initially executed in the normal operational state (T0). When replay inconsistencies, provenance anomalies, or governance-policy violations were detected, workflows transitioned through progressively higher trust states (T1–T3). As shown in Figure 4, replay mismatches triggered transition from T0 to T1, while provenance anomalies and governance-policy violations resulted in escalation to T2 and T3, respectively. Workflows reaching the highest risk state were transitioned into active containment (T4).

FIGURE 4. TRUST ESCALATION AND CONTAINMENT LIFECYCLE

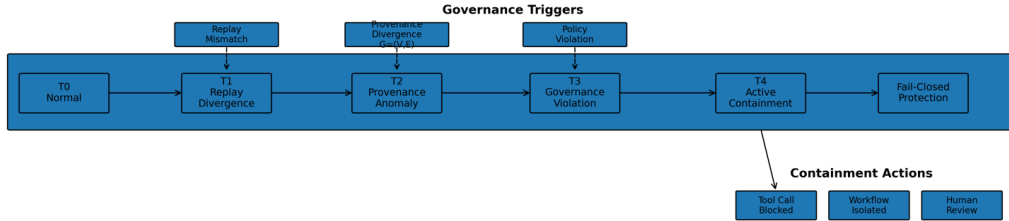


Figure 4: Trust escalation and containment lifecycle.

During containment, governance actions included tool-call blocking, workflow isolation, and escalation to human review. In all evaluated containment scenarios, unsafe workflows were prevented from progressing beyond the containment boundary. No contained workflow resumed execution after containment activation. Across all evaluated attack categories, containment actions were triggered following anomaly detection and trust-state escalation. No unsafe workflow propagation was observed after workflows entered the active containment state. These results indicate that containment decisions remained enforceable throughout the evaluated workflow executions, and that containment actions were successfully applied following governance-trigger events.

3.4 Detection Quality

Detection quality was evaluated using the confusion-matrix metrics defined in Section 3.3.4.

Table 3 summarizes the detection-quality results obtained from a balanced evaluation dataset consisting of 500 normal workflows and 500 adversarial workflows. Across all evaluated workflows, EGA v8 correctly

classified every normal and adversarial execution, yielding 500 true positives and 500 true negatives without any observed false positives or false negatives.

Consequently, Precision, Recall, and F1 Score all reached 100%, while both the False Positive Rate (FPR) and False Negative Rate (FNR) remained at 0%. The results were consistent across all evaluated workflow categories, including Quantity Mutation, Unauthorized Purchase, Tool Injection, Policy Bypass, and Containment Bypass scenarios. Overall, EGA v8 maintained perfect detection performance under the evaluation conditions.

Table 3. Detection Quality Evaluation Results

Metric	Value
Total Workflows	1,000
True Positive (TP)	500
True Negative (TN)	500
False Positive (FP)	0
False Negative (FN)	0
Precision	100%
Recall	100%
F1 Score	100%
False Positive Rate (FPR)	0%
False Negative Rate (FNR)	0%

Note. Detection quality was evaluated using a balanced dataset consisting of 500 normal workflows and 500 adversarial workflows. Metrics were computed from the confusion matrix defined in Section 3.3.4. Across all evaluated workflows, EGA v8 achieved perfect classification performance without false-positive or false-negative detections.

3.5 Live LLM Evaluation

To evaluate governance behavior under realistic language-model interactions, EGA v8 was tested using adversarial workflows generated by live external LLMs, including GPT-4o, Claude 3.5, and Gemini. The evaluation included multiple attack scenarios, **spanning** Quantity Mutation, Tool Injection, Policy Bypass, and Containment Bypass. Figure 5 summarizes the evaluation results. Adversarial workflows generated by the evaluated language models were executed through the EGA v8 runtime-governance stack, where replay validation, provenance analysis, trust-state escalation, and containment enforcement were applied prior to execution approval.

Across all evaluated scenarios, governance violations were successfully detected and contained before unsafe workflow propagation. Detection Rate reached 100%, while Containment Success Rate also reached 100%. No evaluated workflow bypassed governance controls or containment

enforcement, resulting in an Unsafe Propagation Rate of 0%. The observed results were consistent across all tested language models and attack categories.

3.6 Large-Scale Detection Evaluation

FIGURE 5. LIVE LLM ADVERSARIAL EVALUATION RESULTS

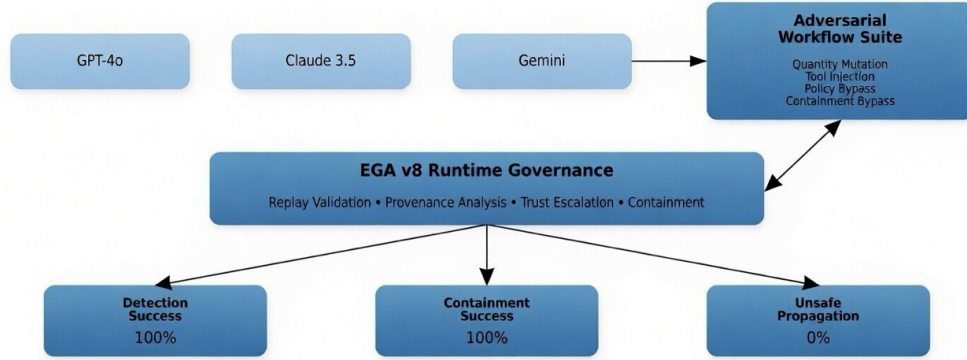


Figure 5: Live LLM adversarial evaluation results.

To evaluate scalability under increasing workflow volumes, detection-quality experiments were conducted at workflow scales of 100, 1,000, and 10,000 instances. The evaluation measured Precision, Recall, F1 Score, False Positive Rate (FPR), and False Negative Rate (FNR) across all scales. Table 4 summarizes the large-scale detection results. Across all evaluated workflow volumes, Precision, Recall, and F1 Score remained at 100%, while both FPR and FNR remained at 0%. No degradation in detection quality was observed as the number of workflow instances increased. The observed results were consistent across all evaluated scales, including the 10,000-workflow benchmark. No additional false-positive or false-negative detections were recorded during the large-scale evaluation campaign.

Table 4. Large-Scale Detection Evaluation Results

Workflow Scale	Precision	Recall	F1 Score	FPR	FNR
100	100%	100%	100%	0%	0%
1,000	100%	100%	100%	0%	0%
10,000	100%	100%	100%	0%	0%

Note. Large-scale detection quality was evaluated at workflow scales of 100, 1,000, and 10,000. Precision, Recall, and F1 Score remained at 100%, while the False Positive Rate (FPR) and False Negative Rate (FNR) remained at 0% across all evaluated scales. Metric definitions are provided in Section 3.3.4.

3.7 Hard-Case Detection Evaluation

To evaluate detection performance under challenging conditions, EGA v8 was tested using hard-case workflows designed to closely resemble legitimate execution behavior. These workflows were constructed to minimize observable differences between normal and adversarial executions, thereby increasing the difficulty of governance-violation detection. Table 5 summarizes the hard-case detection results. Across all evaluated workflows, Precision, Recall, and F1 Score remained at 100%, while both the False Positive Rate (FPR) and False Negative Rate (FNR) remained at 0%. The observed results indicate that detection performance remained unchanged under the hard-case evaluation conditions. No additional false-positive or false-negative detections were recorded during the evaluation campaign.

Table 5. Hard-Case Detection Evaluation Results

Dataset Component / Metric	Value
Evaluation Scale	

Dataset Component / Metric	Value
Total Hard-Case Workflows	1,000
Target Positive (Adversarial)	500
Target Negative (Legitimate)	500
Confusion Matrix Outcomes	
True Positive (TP)	500
True Negative (TN)	500
False Positive (FP)	0
False Negative (FN)	0
Detection Quality Metrics	
Precision	100%
Recall	100%
F1 Score	100%
False Positive Rate (FPR)	0%
False Negative Rate (FNR)	0%

Note. Hard-case detection quality was evaluated using adversarial workflows designed to closely resemble legitimate execution behavior. Precision, Recall, and F1 Score remained at 100%, while the False Positive Rate (FPR) and False Negative Rate (FNR) remained at 0%. Metric definitions are provided in Section 3.3.4.

4 Discussion

4.1 From AI Safety to Execution Governance

Recent advances in autonomous AI systems suggest that safety failures increasingly emerge not from model outputs alone, but from the execution pathways connecting language-model reasoning to external tools and real-world actions. Mythos-like attack scenarios illustrate this progression, where prompt manipulation may lead to unauthorized tool invocation, execution divergence, and ultimately unsafe workflow propagation. Traditional AI-safety approaches primarily rely on prompt filtering, alignment techniques, and static guardrails to prevent unsafe behavior before execution. While these mechanisms remain important, they provide limited visibility into how workflow actions evolve after execution has begun. As autonomous agents gain access to tools, APIs, and external services, governance of execution behavior becomes increasingly critical.

The empirical results presented in Sections 4.1–4.7 suggest that governance controls applied directly at the execution layer can complement existing safety mechanisms. Through deterministic replay, provenance reconstruction, trust-state evaluation, and fail-closed containment, EGA v8 continuously evaluates workflow integrity throughout the execution lifecycle. Collectively, these findings motivate a shift from AI safety as a model-centric problem toward execution governance as a system-level property. Under this perspective, the objective is not only to reduce unsafe outputs, but also to ensure that workflow actions remain traceable, reproducible, auditable, and governable after execution has begun.

4.2 Why Replayability Matters

The transition from AI safety to execution governance raises a fundamental question: how can workflow actions be governed if they cannot be reproduced? Governance requires the ability to reconstruct and verify the sequence of events leading to a particular execution outcome. Without replayability, auditing, anomaly investigation, and policy validation become inherently difficult.

This limitation is evident in many existing agent-runtime environments, where execution records are primarily used for logging and post hoc inspection. While such records provide visibility into what occurred, they do not necessarily enable deterministic reconstruction of how workflow actions were produced. Consequently, execution histories often function as observational artifacts rather than verifiable evidence. In contrast, EGA

v8 treats replayability as a governance primitive. By reconstructing workflow execution through deterministic replay artifacts, the framework enables execution outcomes to be independently reproduced and verified. The replay-validation results in Section 4.1 demonstrated a replay rate of 100%, with replay-root fingerprints and workflow-state transitions remaining identical between original and reconstructed executions.

Replayability also serves as the foundation for higher-level governance mechanisms. Provenance reconstruction, trust-state evaluation, and containment decisions all depend on reproducible execution histories. Without deterministic replay, these controls cannot reliably establish execution integrity. These observations suggest that replayability is not merely a debugging or observability feature. Rather, it transforms execution histories from observational logs into verifiable governance evidence, forming the bedrock upon which execution governance can be established.

4.3 Provenance as Operational Explainability

The replayability foundation discussed in Section 5.2 raises a second governance question: once an execution can be reproduced, how can its behavior be explained? Traditional explainable-AI approaches focus on interpreting model internals, such as attention patterns, feature importance, or model weights. While these techniques provide insight into model behavior, they offer limited visibility into how autonomous agents interact with tools, services, and workflows during execution. The provenance-validation results in Section 4.2 suggest an alternative perspective. Rather than explaining why a model generated a particular output, EGA v8 focuses on explaining how a workflow action emerged through observable execution events. By representing workflow execution as a provenance graph, the framework reconstructs causal relationships among tool invocations, workflow transitions, and governance decisions.

Under this approach, explainability becomes an operational property rather than a model property. Provenance DAGs enable investigators to trace workflow outcomes back to the actions and decision points that produced them, providing visibility into execution causality. This capability is particularly valuable in autonomous-agent environments, where governance failures often arise from interactions among multiple execution components rather than from a single model response. The empirical results in Section 4.2 demonstrate that these causal pathways can be systematically audited and verified after execution. Consequently, provenance functions not merely as a tracing mechanism, but as a practical and verifiable form of operational explainability for autonomous AI systems.

4.4 Comparative Analysis with Existing Agent Runtimes

Figure 6. Comparative Runtime Governance Outcomes

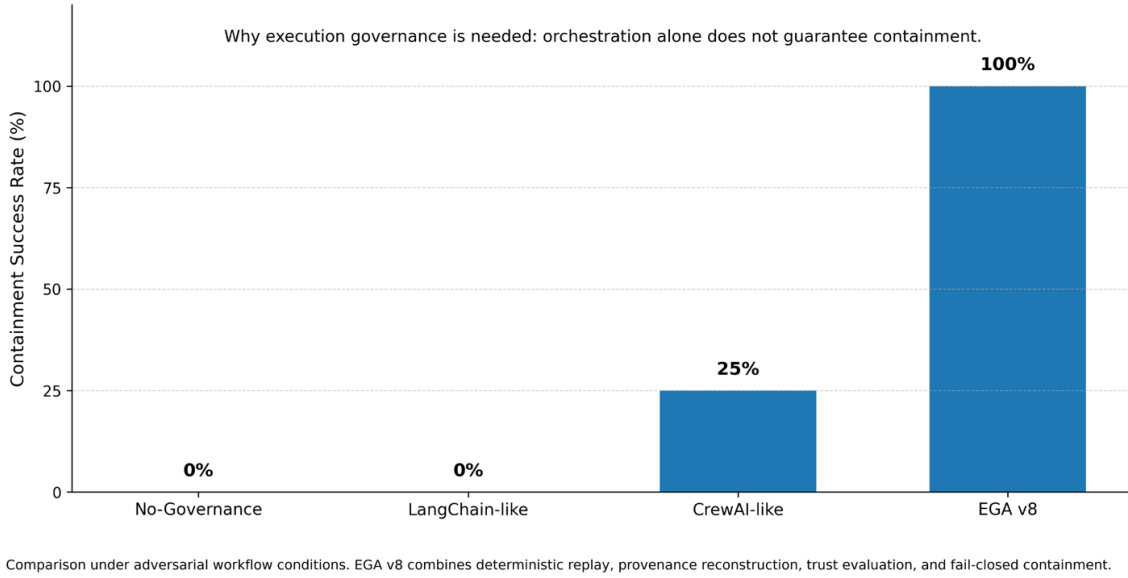


Figure 6: Comparative runtime governance outcomes.

The necessity of replayability and provenance reconstruction raises a pivotal question: to what extent can existing agent-runtime frameworks achieve these governance capabilities? Figure 6 contrasts containment outcomes across representative autonomous-agent architectures under adversarial conditions. While the No-Governance baseline and LangChain-like runtimes failed to impede unsafe workflow propagation (0% success), and CrewAI-like runtimes exhibited limited containment capability at 25%, EGA v8 consistently maintained a 100% containment success rate across all evaluated scenarios. This performance divergence is rooted in architectural design rather than model capability. Conventional orchestration-oriented runtimes focus primarily on coordinating agent interactions, tool invocations, and workflow sequencing. Although these frameworks provide valuable orchestration functionality, they generally lack deterministic replay, provenance-aware reconstruction, trust-state evaluation, and fail-closed containment as native governance primitives.

In contrast, EGA v8 continuously enforces execution integrity throughout the runtime lifecycle. By anchoring governance in replay validation, provenance analysis, trust escalation, and containment enforcement, the framework systematically isolates anomalous execution paths before unsafe propagation can occur. These comparative findings suggest that execution governance constitutes a distinct architectural layer beyond conventional orchestration, providing the governance mechanisms required to maintain workflow integrity in autonomous AI systems.

4.5 Why Perfect Detection Was Observed

The perfect detection and containment results reported in Sections 4.4–4.7 may initially appear surprising given the complexity of autonomous-agent environments. A natural question is whether these outcomes reflect characteristics of the evaluation datasets rather than the governance architecture itself. The extended validation results provide evidence against this interpretation. Section 4.6 demonstrated stable detection performance across workflow volumes ranging from 100 to 10,000 executions, while Section 4.7 evaluated hard-case workflows intentionally designed to resemble legitimate execution behavior. Across both evaluations, Precision, Recall, and F1 Score remained at 100%, while False Positive and False Negative rates remained at 0%.

One explanation for these results is that EGA v8 does not rely primarily on probabilistic anomaly scoring or heuristic pattern matching. Instead, governance decisions are derived from deterministic execution ar-

tifacts, including replay validation, provenance reconstruction, workflow-state consistency, and trust-state evaluation. Consequently, detection is driven by observable execution divergence rather than statistical estimation of behavioral risk. This distinction is important because execution divergence produces explicit governance evidence. Replay validation identifies inconsistencies between observed and reconstructed execution paths, while provenance analysis exposes causal deviations within workflow histories. As a result, governance decisions are grounded in verifiable execution artifacts rather than confidence thresholds alone. These observations suggest that the perfect detection results reported in this study are more appropriately interpreted as a consequence of deterministic governance validation than as an artifact of dataset simplicity.

Collectively, the findings support the view that replay-based governance, provenance-aware reconstruction, and execution-level validation can serve as effective mechanisms for maintaining workflow integrity in autonomous AI systems.

4.6 Limitations and Future Work

The findings presented in this study should be interpreted within the scope of the proposed execution-governance framework. EGA v8 is designed to govern workflow execution through replay validation, provenance reconstruction, trust-state evaluation, and fail-closed containment. Accordingly, the framework addresses risks associated with execution-layer governance rather than all possible security threats affecting AI systems.

Several threat categories remain outside the scope of the present work. These include operating-system compromise, network-level attacks, hardware-level tampering, and infrastructure-security threats affecting the underlying execution environment. Likewise, model-centric attacks such as training-data poisoning, model-weight extraction, and parameter manipulation are not explicitly addressed by the proposed framework.

EGA v8 is not intended to replace existing security mechanisms or model-safety techniques. Rather, it complements these approaches by introducing governance capabilities at the execution layer, operating alongside model safety, system security, and operational monitoring.

Future work will extend the evaluation to broader classes of autonomous-agent architectures, multi-agent environments, and large-scale production deployments. Additional research may also explore tighter integration between execution-governance mechanisms and model-level safety systems.

These limitations do not diminish the central contribution of this work; rather, they define the operational scope within which EGA v8 is intended to function. Collectively, the findings suggest that replay-based governance, provenance-aware reconstruction, trust-state evaluation, and fail-closed containment provide a practical foundation for maintaining workflow integrity within autonomous AI systems.

5 Conclusion

The findings of this study suggest that maintaining trustworthy autonomous AI systems requires governance mechanisms beyond model-centric safety controls alone. As autonomous agents increasingly interact with tools, services, and external environments, workflow integrity becomes a runtime property that must be continuously verified rather than implicitly assumed. EGA v8 addressed this challenge by introducing an execution-governance framework grounded in deterministic replay, provenance-aware reconstruction, trust-state evaluation, and fail-closed containment. Across large-scale and hard-case evaluations, the framework demonstrated the feasibility of enforcing governance constraints directly at the execution layer.

More broadly, this work argues that the continued scaling of autonomous-agent ecosystems may expose limitations in safety approaches that rely primarily on token-level monitoring and probabilistic alignment mechanisms. As workflow complexity and execution density increase, governance mechanisms operating directly on execution artifacts may provide a more scalable and verifiable foundation for maintaining workflow integrity. In this context, deterministic replay and provenance validation offer an alternative governance model that operates independently of token-level abstraction.

Ultimately, replayability, provenance, and execution-level validation represent not merely implementation features, but foundational governance primitives for autonomous AI systems. As agent deployments continue to expand, execution governance may emerge as a critical complementary control plane alongside model safety and system security. Under this perspective, the future challenge of the AI ecosystem is not only building more capable agents, but ensuring that their actions remain traceable, reproducible, auditable, and governable throughout the execution lifecycle.